

Nama : Muhammad Rafif Permana Putra

NPM : 51422112

Kelas : 4IA09

1. Peran Server dan Client

Dalam komunikasi jaringan, kedua entitas ini memiliki tanggung jawab yang berbeda namun saling bergantung:

- **Server:** Berfungsi sebagai pusat kendali atau "host". Server harus selalu aktif (mendengarkan/listening) menunggu permintaan koneksi. Dalam aplikasi chat, server bertugas menerima pesan dari satu client dan meneruskannya (broadcasting) ke client lain yang terhubung.
- **Client:** Adalah antarmuka pengguna yang menginisiasi koneksi ke server menggunakan alamat IP dan nomor port tertentu. Client mengirimkan pesan ke server dan menerima pesan dari pengguna lain melalui server.

2. Langkah-langkah Pembuatan Program

a. Inisialisasi Socket dan Koneksi

Proses dimulai dengan pembuatan objek socket pada kedua belah pihak. Socket adalah pintu gerbang (endpoint) komunikasi antar perangkat.

1. **Server Bind & Listen:** Server menentukan alamat IP (biasanya 0.0.0.0 untuk menerima dari semua interface) dan port (misalnya 5000). Server kemudian menjalankan fungsi `listen()` untuk masuk ke mode siaga.
2. **Client Connect:** Client memanggil fungsi `connect()` dengan menysasar alamat IP server dan port yang sama.
3. **Acceptance:** Server menerima permintaan tersebut dengan fungsi `accept()`, yang akan menghasilkan objek socket baru khusus untuk berkomunikasi dengan client tersebut.

b. Pengiriman dan Penerimaan Data

Setelah koneksi terjalin, data dikirim dalam bentuk **byte**.

- **Encoding:** Pesan teks dari pengguna harus diubah menjadi byte (biasanya menggunakan format UTF-8) sebelum dikirim melalui fungsi `send()`.
- **Decoding:** Penerima mengambil data menggunakan fungsi `recv()` dan mengubah kembali byte tersebut menjadi teks string agar bisa dibaca manusia.

c. Penanganan Banyak Client (Multithreading)

Secara default, program berjalan secara sekuensial (satu persatu). Jika satu client sedang mengirim pesan, server tidak bisa melayani client lain.

Untuk mengatasinya, kita menggunakan **Multithreading**:

1. Setiap kali ada client baru yang terhubung (accept), server akan membuat satu **Thread** baru khusus untuk client tersebut.
2. Thread tersebut bertugas menjalankan *loop* terus-menerus untuk mendengarkan pesan dari client itu saja.
3. Dengan cara ini, main thread server tetap bebas untuk menerima koneksi dari client baru lainnya.

3. Alur Logika dalam Kode (Contoh Python)

Sisi Server:

1. Buat socket: `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`.
2. Ikat ke alamat: `server.bind((IP, PORT))`.
3. Tunggu koneksi: `server.listen()`.
4. Gunakan *loop* utama untuk `accept()`.
5. Setiap koneksi masuk, masukkan ke dalam daftar (list) client dan jalankan fungsi `handle_client` di thread baru.
6. Fungsi broadcast: Jika pesan diterima dari satu thread, kirimkan pesan tersebut ke semua socket yang ada di daftar client.

Sisi Client:

1. Buat socket dan hubungkan ke server.
2. Jalankan dua thread sekaligus:
 - **Thread 1:** Menunggu input pengguna di terminal/GUI lalu mengirimkannya ke server.
 - **Thread 2:** Terus-menerus mendengarkan pesan masuk dari server untuk ditampilkan di layar.

4. Keamanan dan Penanganan Error

- **Exception Handling:** Gunakan blok try-except untuk menangani client yang tiba-tiba memutus koneksi (misalnya kabel LAN terlepas) agar server tidak *crash*.
- **Buffer Size:** Tentukan ukuran data yang diterima (misalnya 1024 byte). Jika pesan terlalu besar, ia harus dipecah atau ditangani secara khusus.
- **Firewall:** Pastikan port yang digunakan telah diizinkan oleh Firewall Windows/Linux agar paket data tidak diblokir saat mencoba masuk ke server.